Overview

`sunlineEphem` computes the Sun direction unit vector relative to the spacecraft in the body frame based on ephemeris data. This direction vector can be used as a reference for downstream guidance and control modules.

The module layer (`SunlineEphem`) implements the SysModel interface, wiring messages and managing module lifecycle. The algorithm (`SunlineEphemAlgorithm`) contains the stateless computation that can be used stand-alone when the caller provides position and attitude data directly.

Module Inputs and Outputs

Module inputs:

Table 1: Module Input Messages

Msg Variable Name	Msg Type	Description
sunPositionInMsg	`EphemerisMsgF32Payload`	Read every `updateState()`. Contains the Sun's position expressed in the inertial frame.
scPositionInMsg	`NavTransMsgF32Payload`	Read every `updateState()`. Contains the spacecraft's position expressed in the inertial frame.
scAttitudeInMsg	`NavAttMsgF32Payload`	Read every `updateState()`. Contains the spacecraft's attitude (MRPs), used to rotate the Sun vector from the inertial frame to body frame.

Module output:

Table 2: Module Output Messages

Msg Variable Name	Msg Type	Description
navStateOutMsg	`NavAttMsgF32Payload`	Ephemeris-based Sun direction unit vector expressed in the body frame, written to `vehSunPntBdy`.

Algorithm Assumptions

- All input position vectors must be expressed in the same inertial reference frame.
- The algorithm returns a unit direction vector for distinct positions. For colocated positions, the zero vector is returned as a sentinel when no direction can be computed. Downstream consumers should check for this case.

Algorithm Description

Given Sun position ${}^N\boldsymbol{r}_{S/N}$, spacecraft position ${}^N\boldsymbol{r}_{B/N}$, and spacecraft attitude MRPs $\boldsymbol{\sigma}_{BN}$:

1. Compute the Sun position relative to the spacecraft in the inertial frame:

$$ {}^N\boldsymbol{r}_{S/B} = {}^N\boldsymbol{r}_{S/N} - {}^N\boldsymbol{r}_{B/N} $$

2. If $\|{}^N\boldsymbol{r}_{S/B}\| \leq \epsilon$, return the zero vector. Otherwise, normalize the relative vector, build the DCM from the spacecraft attitude MRPs, and rotate the unit direction into the body frame:

$$ {}^B\hat{\boldsymbol{r}}_{S/B} = \text{normalize}\left([BN]\,{}^N\hat{\boldsymbol{r}}_{S/B}\right) $$

Figure 1: SunlineEphem computes the inertial-frame Sun direction vector ${}^N\boldsymbol{r}_{S/B}$.

Module vs. Algorithm Usage

- Module (`SunlineEphem`, SysModel): use when operating inside the messaging framework. Connect `sunPositionInMsg`, `scPositionInMsg`, and `scAttitudeInMsg`, call `InitializeSimulation()/ExecuteSimulation()`, and read `navStateOutMsg`.
- Algorithm (`SunlineEphemAlgorithm`): use when you have position and attitude data directly. Call the static `update(r_SN_N, r_BN_N, sigma_BN)` method, which returns the body-frame Sun direction vector.

Test Description and Success Criteria

The Python integration test is located in `fswAlgorithms/attDetermination/sunlineEphem/_UnitTest/test_` This test verifies that the module computes the correct Sun direction vector expressed in the body frame. In this unit test, the Sun inertial position is set at the origin by `sunData.r_BdyZero_N = [0.0, 0.0, 0.0]`. The spacecraft attitude is set to zero using `vehAttData.sigma_BN = [0.0, 0.0, 0.0]`, which corresponds to an identity rotation. This implies that the inertial and body frames are aligned. The spacecraft's inertial position is iterated through a set of unit length vectors, with a special case where the spacecraft and sun position vectors are identical. For each case, the simulation will execute and the Sun direction vector will be recorded. The truth vectors are defined in the python test, where they are compared with the estimated module output. The test is considered successful if all the output values of the estimated and truth values are identical within a tolerance of 10^{-12}, including outputting a zero vector for the special case where the relative vector magnitude is zero.